

Face Recognition Dataset Report

Deep Learning Tasks 1 to 8

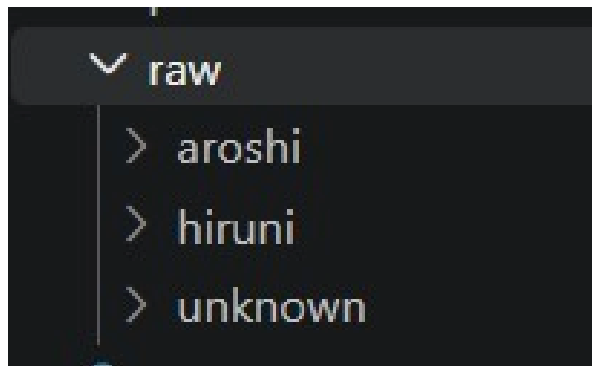
Task 1 - Dataset Structure & Configuration

1. Why is the unknown class important?

The unknown class is important because the system should distinguish registered friends from people who are not in the known classes. Without an unknown class, the model may be forced to classify an unfamiliar person as one of the known friends.

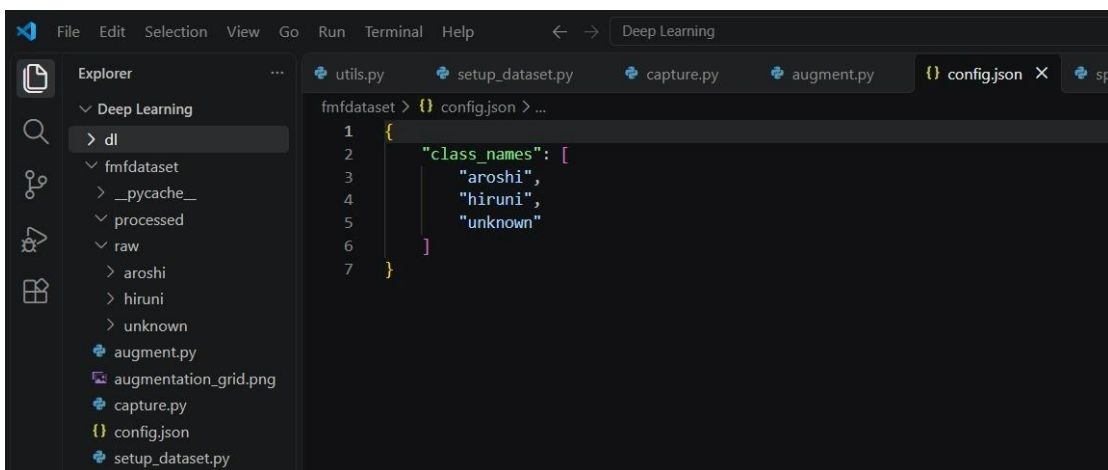
2. Dataset structure

The dataset contains three classes: aroshi, hiruni and unknown. The raw images are stored inside the corresponding class folders.



3. config.json

The class order used in config.json is: ["aroshi", "hiruni", "unknown"].



4. Why is class order important?

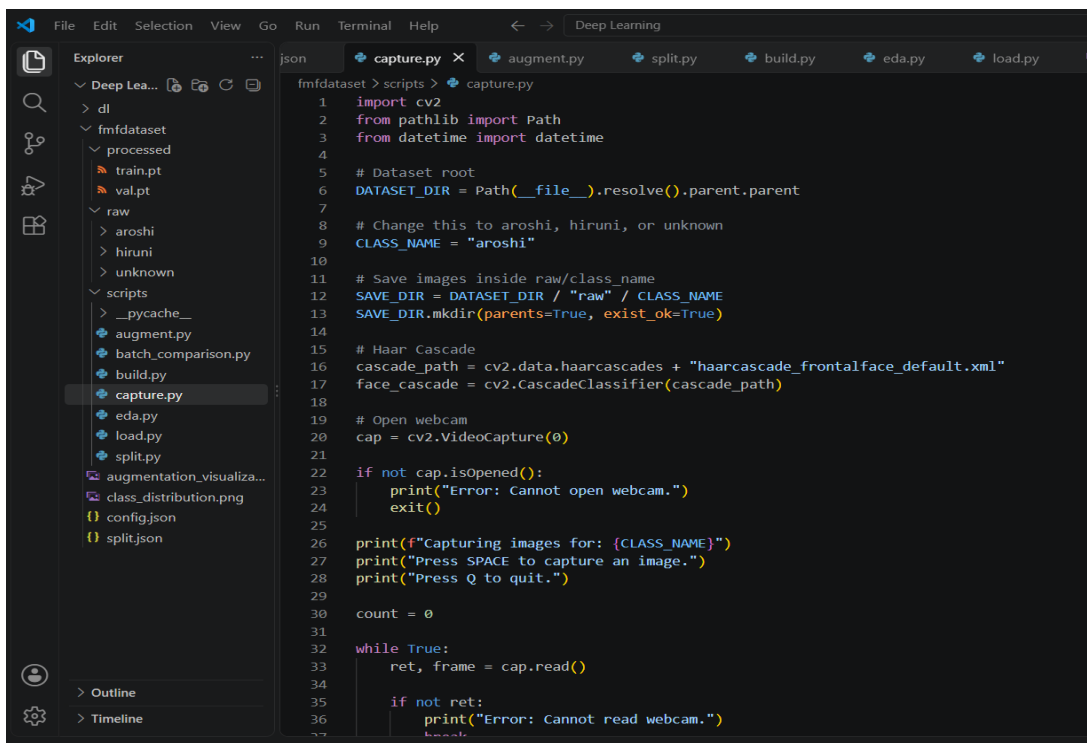
Class order is important because each class is assigned a numerical label. The same class-to-label mapping must be used during dataset creation, training, validation and prediction. If the order changes, the numerical labels may represent different people and predictions can be interpreted incorrectly.

Task 2 - Face Data Collection

1. Capture process

OpenCV was used to capture face images from a webcam. A Haar Cascade classifier was used to detect faces. When a face was detected, a rectangle was drawn around it. Pressing the Space key captured the cropped face with 20% padding and saved it using a timestamp-based filename. Pressing q stopped the capture process.

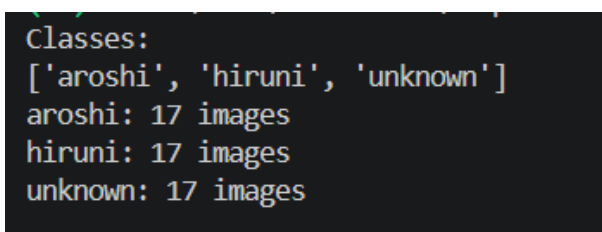
Script: scripts/capture.py

A screenshot of a code editor window titled 'Deep Learning'. The 'Explorer' panel on the left shows a file tree with folders 'Deep Learning', 'fmfdataset', and 'scripts'. The 'scripts' folder is expanded, showing files like 'capture.py', 'eda.py', 'load.py', and 'split.py'. The 'capture.py' file is selected and its code is visible in the main editor. The code imports cv2, Path, and datetime. It defines a dataset root, a class name 'aroshi', and a save directory. It uses a Haar Cascade classifier to detect faces from a webcam. When a face is detected, it captures the image with 20% padding and saves it with a timestamp-based filename. The process continues until the user presses 'q' to quit.

```
1 import cv2
2 from pathlib import Path
3 from datetime import datetime
4
5 # Dataset root
6 DATASET_DIR = Path(__file__).resolve().parent.parent
7
8 # Change this to aroschi, hiruni, or unknown
9 CLASS_NAME = "aroschi"
10
11 # Save images inside raw/class_name
12 SAVE_DIR = DATASET_DIR / "raw" / CLASS_NAME
13 SAVE_DIR.mkdir(parents=True, exist_ok=True)
14
15 # Haar Cascade
16 cascade_path = cv2.data.harcascades + "haarcascade_frontalface_default.xml"
17 face_cascade = cv2.CascadeClassifier(cascade_path)
18
19 # Open webcam
20 cap = cv2.VideoCapture(0)
21
22 if not cap.isOpened():
23     print("Error: Cannot open webcam.")
24     exit()
25
26 print(f"Capturing images for: {CLASS_NAME}")
27 print("Press SPACE to capture an image.")
28 print("Press Q to quit.")
29
30 count = 0
31
32 while True:
33     ret, frame = cap.read()
34
35     if not ret:
36         print("Error: Cannot read webcam.")
37         exit()
```

2. Data collection log

The final raw dataset contains 17 images for each class: aroschi – 17, hiruni – 17, unknown – 17. Total = 51 original images.

A screenshot of a terminal window showing the output of a command. The output lists the classes and the number of images collected for each class.

```
Classes:
['aroschi', 'hiruni', 'unknown']
aroschi: 17 images
hiruni: 17 images
unknown: 17 images
```

3. scaleFactor

scaleFactor controls how much the image size is reduced between detection stages. A smaller value searches at more scales and can improve detection but takes more time. A larger value is faster but may miss some faces.

4. minNeighbors

minNeighbors controls how many nearby detections are required before a region is considered a face. Increasing it reduces false detections but may miss some faces. Decreasing it detects more faces but can produce more false positives.

5. Three raw-image problems

- Poor or uneven lighting.
- Motion blur or camera blur.
- Different face poses or partial occlusion.

Task 3 - Data Augmentation

1. Augmentations used

Augmentation	Purpose
Horizontal Flip	Simulates mirrored face orientation
Rotation	Simulates small head tilts
Brightness	Simulates different lighting
Contrast	Simulates camera/exposure differences
Gaussian Blur	Simulates motion or focus blur
Perspective	Simulates changes in viewing angle



2. Augmentation calculation for this dataset

Task 4 produced 40 training images and 11 validation images. Each training image was randomly augmented 6 times. Therefore, $40 \times 6 = 240$ training samples. Validation images were not randomly augmented.

3. Most realistic augmentation

Horizontal flipping and small rotations appeared to be the most realistic because these changes can naturally occur when a person's face is viewed from slightly different orientations.

4. Least realistic augmentation

Perspective transformation was the least realistic when the distortion became too strong because it produced face shapes that are unlikely to occur naturally in normal camera conditions.

5. Augmentation factor risks

Too low: the model may not receive enough variation and could overfit to the original training images.
Too high: excessive augmentation can create unrealistic samples and may increase training time and storage requirements.

Task 4 - Train/Validation Split

1. Splitting method

The dataset was split into 80% training and 20% validation data using a fixed random seed of 42. Stratification was used to maintain approximately the same class proportions in both sets.

2. Actual dataset split

Total images = 51. Training images = 40. Validation images = 11.

```
(dl) PS D:\NIBM\HNDDS 25.2\Deep Learning\fmfdataset> python scripts\split.py
Total images      : 51
Training images   : 40
Validation images : 11

Split file saved to:
D:\NIBM\HNDDS 25.2\Deep Learning\fmfdataset\split.json
(dl) PS D:\NIBM\HNDDS 25.2\Deep Learning\fmfdataset> 
```

3. Why validation data isn't augmented

Validation data should represent the real data distribution used for evaluation. Random augmentation could make validation results inconsistent and make it difficult to measure actual performance. Resizing and normalization are acceptable because they are preprocessing operations rather than random data augmentation.

4. Smile vs neutral problem

If all smiling images were placed in the training set and all neutral images in the validation set, the model could learn expression-specific patterns instead of facial identity. To avoid this, different expressions and poses should be distributed across both training and validation sets.

Task 5 - Build .pt Dataset

1. Dataset creation

The original training images were loaded and each training image was randomly augmented six times. The resulting images were resized to 224×224 and converted into PyTorch tensors. Validation images were resized to 224×224 and converted to tensors without random augmentation.

2. Actual output

Training: torch.Size([240, 3, 224, 224]); Labels: torch.Size([240]). Validation: torch.Size([11, 3, 224, 224]); Labels: torch.Size([11]).

```
Train:
Images shape: torch.Size([240, 3, 224, 224])
Labels shape: torch.Size([240])
Number of samples: 240

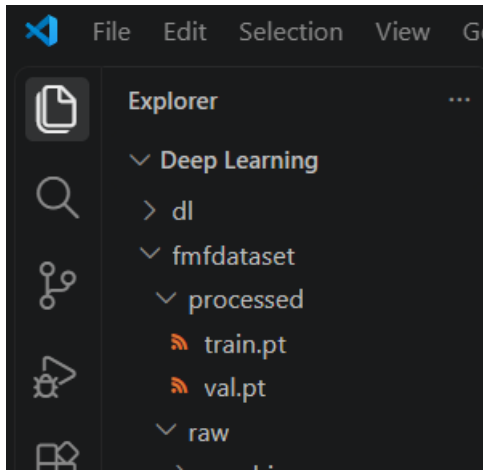
Validation:
Images shape: torch.Size([11, 3, 224, 224])
Labels shape: torch.Size([11])
Number of samples: 11
```

3. Memory calculation

Image shape = $3 \times 224 \times 224$; data type = float32; bytes per float = 4. Memory per image = $3 \times 224 \times 224 \times 4 = 602,112$ bytes ≈ 0.574 MiB. For 240 training images, the image tensor memory is $240 \times 602,112 = 144,506,880$ bytes ≈ 137.79 MiB (excluding labels and metadata).

4. Why .pt instead of JPEG?

.pt files can store tensors and labels together and can be loaded directly by PyTorch. This avoids repeatedly converting JPEG images into tensors during training. However, .pt files can require more storage, are less convenient to inspect manually, and are mainly designed for PyTorch workflows.

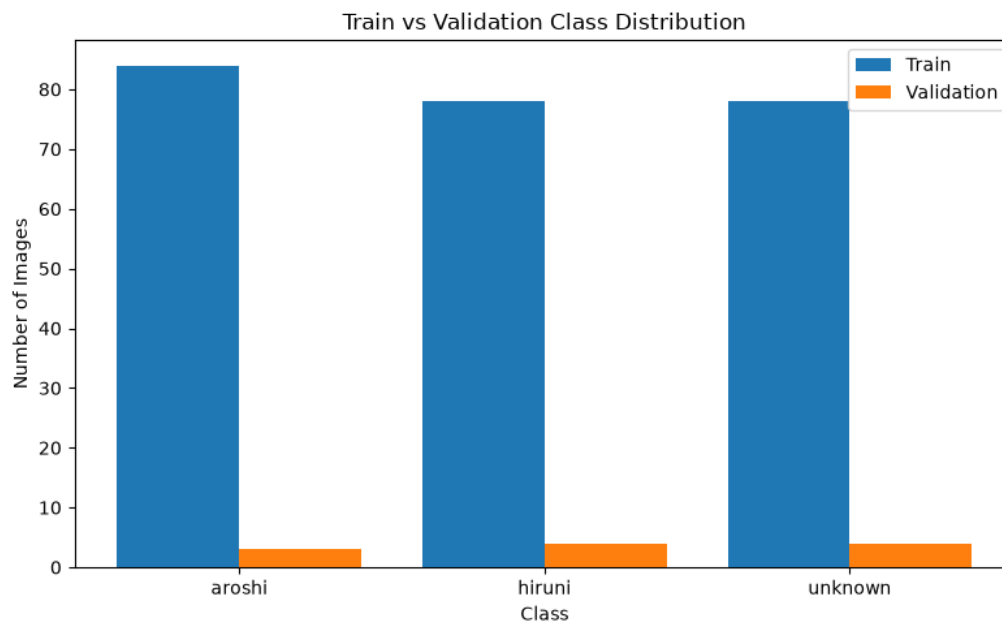


Task 6 -Dataset Analysis

1. Dataset shapes

Training: [240, 3, 224, 224]. Validation: [11, 3, 224, 224].

2. Class distribution chart



3. Random image grid

Random 16 images.png generated by eda.py.

4. Is the dataset balanced?

The original raw dataset is balanced because each of the three classes contains 17 images. After the 80/20 stratified split, the class counts are kept approximately balanced. The training set is then expanded by applying six augmentations to each training image.

5. Why store paths?

The paths field stores the source path of each image. It is useful for tracing a tensor back to the original image when debugging incorrect labels, investigating misclassified samples, or checking problematic training examples.

Task 7 -Dataset & DataLoader

1. Dataset class

Script: scripts/load.py. It implements `__init__()`, `__len__()` and `__getitem__()`.

2. DataLoader output

With batch size 8, one training batch has image shape [8, 3, 224, 224] and label shape [8].

```
Train dataset size:
240

Validation dataset size:
11

Batch size:
8

One training batch:
Images shape: torch.Size([8, 3, 224, 224])
Labels shape: torch.Size([8])
```

3. ImageNet normalization

ImageNet normalization uses mean values [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225] for the RGB channels. These values are commonly used with pretrained computer vision models trained on ImageNet.

4. Batch size 8 vs 64

Factor	Batch 8	Batch 64
Memory	Lower	Higher
Speed	Usually lower throughput	Usually higher throughput
Gradient	More noisy	Smoother
Hardware requirement	Lower	Higher
Suitable for limited RAM/VRAM	Better	Less suitable

```
Number of batches: 4
Time taken: 0.0901 seconds
COMPARISON

Batch size 8:
- Uses less memory
- More batches
- Suitable for systems with lower memory

Batch size 64:
- Uses more memory
- Fewer batches
- Can process more samples at once
- Requires more memory
```

Task 8 - Ethics & Production

Ethical guidelines

- Obtain consent from people whose faces are collected.
- Store face data securely and restrict access.
- Delete the data when it is no longer required.

Failure scenarios

- Poor lighting or blurry images.
- Unknown person incorrectly classified as a known friend.
- Different pose, occlusion, or appearance causing recognition failure.

Production improvements

- Collect a larger and more diverse dataset.
- Use confidence thresholds so uncertain predictions can be rejected.
- Add security, monitoring and regular evaluation.

GitHub Link -

<https://github.com/hiruniwathsala88-source>

